

FEATURE: Attachment Relevance Checker

(Project: CyberNexus – Jira Compliance Agent)

1 Objective

For each Jira ticket:

1. Fetch attachments (PDF, image, text, etc.)
 2. Extract full content (no truncation at extraction stage)
 3. Compare attachment content with:
 - Ticket summary
 - Ticket description
 - Ticket comments
 4. If attachment is NOT relevant:
 - Add entry to compliance_gaps
 - Mark ticket as **Dissatisfied**
-

2 High-Level System Flow

Fetch Ticket



Fetch Attachments



Download Files



Extract Full Text



Compute File Hash



Check Database (Skip if unchanged)



Chunk Text



Embedding Similarity Check



If Not Relevant → Add Deficiency



Store Results in DB

3 Jira Fetch Modifications

3.1 Add Attachment Field in Fetch

Modify fields_list:

```
fields_list = [  
    "summary", "description", "status", "comment",  
    "priority", "assignee", "reporter",  
    "created", "updated", "duedate",  
    "attachment"  
]
```

3.2 Store Attachment Metadata

Inside extract_ticket_doc():

```
attachments = fields.get("attachment", [])
```

```
"attachments": [  
    {  
        "attachment_id": att.get("id"),  
        "filename": att.get("filename"),
```

```

        "mimeType": att.get("mimeType"),
        "content": att.get("content") # download URL
    }
    for att in attachments
]

```

Now each ticket includes attachment metadata.

4 Secure Attachment Download

Jira attachment URLs require authentication.

```

def download_attachment(url, email, token):
    response = requests.get(
        url,
        auth=HTTPBasicAuth(email, token),
        timeout=60
    )
    response.raise_for_status()
    return response.content

```

5 Full Content Extraction (NO TRUNCATION)

5.1 PDF Extraction

```

def extract_pdf_text(file_bytes):
    import pdfplumber
    import io

    full_text = ""
    with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
        for page in pdf.pages:

```

```
full_text += (page.extract_text() or "") + "\n"

return full_text
```

5.2 Image OCR (Screenshots)

System dependency: Tesseract must be installed.

```
def extract_image_text(file_bytes):

    from PIL import Image

    import pytesseract

    import io

    image = Image.open(io.BytesIO(file_bytes))

    return pytesseract.image_to_string(image)
```

5.3 Text Files

```
def extract_text_file(file_bytes):

    return file_bytes.decode("utf-8", errors="ignore")
```

5.4 Unified Extractor

```
def extract_attachment_content(attachment, email, token):

    file_bytes = download_attachment(attachment["content"], email, token)

    mime = attachment.get("mimeType", "").lower()

    if "pdf" in mime:

        return extract_pdf_text(file_bytes)

    elif "image" in mime:

        return extract_image_text(file_bytes)
```

```
elif "text" in mime:
    return extract_text_file(file_bytes)

else:
    return ""
```

6 IMPORTANT: Do NOT Send Full Attachment to LLM

LLMs have token limits.

We must separate:

1. Extraction
2. Storage
3. Relevance evaluation

We use embeddings instead of sending entire document to LLM.

7 Relevance Evaluation (Embedding-Based)

7.1 Process

1. Generate embedding for ticket content (summary + description + comments)
 2. Split full attachment text into chunks
 3. Generate embedding for each chunk
 4. Compute cosine similarity
 5. Use max similarity as relevance score
-

7.2 Chunking

```
def chunk_text(text, chunk_size=2000):
    chunks = []
    for i in range(0, len(text), chunk_size):
```

```
chunks.append(text[i:i+chunk_size])  
return chunks
```

7.3 Similarity Logic

```
ticket_embedding = embed(ticket_text)  
  
chunks = chunk_text(full_attachment_text)  
  
max_similarity = 0  
  
for chunk in chunks:  
    chunk_embedding = embed(chunk)  
    similarity = cosine_similarity(ticket_embedding, chunk_embedding)  
    max_similarity = max(max_similarity, similarity)  
  
is_relevant = max_similarity >= 0.65  
Threshold: 0.65 (configurable)
```

8 Compliance Rule Update

After similarity evaluation:

If is_relevant == False:

```
missing_fields.append("Attachment not relevant to ticket content")
```

Ticket automatically becomes Dissatisfied.

9 Edge Case Handling

Case 1 — No Attachment (When Required)

If severity = High or Critical AND no attachment:

Add deficiency:

"Mandatory attachment missing for high severity ticket"

Case 2 — Large Attachments

We extract full content.

But chunk for embedding processing.

No truncation at extraction stage.

Case 3 — Multiple Attachments

Process each separately.

If ANY attachment is relevant → ticket passes attachment check.

If ALL attachments irrelevant → ticket fails.

10 Performance Strategy

Attachments are heavy.

For 1000 tickets:

- 1000 PDFs = heavy download
- OCR is slow
- Embeddings cost increases

Solution:

- Cache extracted text in DB
 - Store file hash
 - Only reprocess if changed
-

11 Database Architecture (Multi-Tenant Safe)

Existing Tables

- tickets
- ticket_reports

1 tickets Table

```
CREATE TABLE tickets (  
    tenant_id TEXT NOT NULL,  
    ticket_key TEXT NOT NULL,  
    project_key TEXT NOT NULL,  
    jira_updated TIMESTAMP,  
    ticket_hash TEXT NOT NULL,  
    attachment_combined_hash TEXT,  
    last_analyzed TIMESTAMP,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    PRIMARY KEY (tenant_id, ticket_key)  
);
```

Reanalyze if:

- ticket_hash changed
OR
- attachment_combined_hash changed

2 attachments Table (NEW)

```
CREATE TABLE attachments (  
    tenant_id TEXT NOT NULL,  
    ticket_key TEXT NOT NULL,  
    attachment_id TEXT NOT NULL,  
    filename TEXT,  
    mime_type TEXT,  
    file_hash TEXT NOT NULL,  
    extracted_text TEXT,  
    relevance_score FLOAT,  
    is_relevant BOOLEAN,
```



```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
PRIMARY KEY (tenant_id, attachment_id),  
  
FOREIGN KEY (tenant_id, ticket_key)  
  REFERENCES tickets(tenant_id, ticket_key)  
  ON DELETE CASCADE  
);
```

3 Row-Level Security

```
ALTER TABLE attachments ENABLE ROW LEVEL SECURITY;  
  
CREATE POLICY tenant_isolation_attachments  
ON attachments  
USING (tenant_id = current_setting('app.current_tenant'));  
Ensures multi-tenant isolation.
```

12 Attachment Hashing (Skip Reprocessing)

Before extracting:

```
SELECT file_hash FROM attachments  
WHERE tenant_id = :tenant  
AND attachment_id = :id;
```

If same hash → skip extraction + similarity.

Compute combined hash:

```
combined_attachment_hash = sha256(  
  attachment1_hash + attachment2_hash + ...  
)
```

Store in tickets table.

13 Ticket_reports Update

No schema change required.

But store in JSON:

```
{  
  "attachment_check": {  
    "total_attachments": 2,  
    "relevant": 1,  
    "irrelevant": 1  
  }  
}
```

14 Optimization

Add index:

```
CREATE INDEX idx_attachments_ticket  
ON attachments (tenant_id, ticket_key);
```

15 Final System Behavior

System now supports:

- ✓ Full attachment extraction
- ✓ OCR for images
- ✓ PDF parsing
- ✓ Embedding-based relevance
- ✓ Multi-tenant isolation
- ✓ Hash-based reprocessing
- ✓ No duplicate analysis
- ✓ Scalable to 1000+ tickets

utils/attachment.py – Attachment download, extraction, and relevance checking.

UPDATED VERSION:

- Sentence-aware semantic chunking
- embed_query (ticket) vs embed_documents (chunks)
- Batch embedding for performance
- No unsafe hard truncation
- Lowered default threshold (0.45)

"""

```
from __future__ import annotations
```

```
import hashlib
```

```
import io
```

```
import logging
```

```
import os
```

```
import re
```

```
from typing import Optional
```

```
import numpy as np
```

```
import requests
```

```
from requests.auth import HTTPBasicAuth
```

```
logger = logging.getLogger(__name__)
```

```
# -----
```

```
# Config
```

```
# -----
```

```
RELEVANCE_THRESHOLD: float = float(  
    os.getenv("ATTACHMENT_RELEVANCE_THRESHOLD", "0.45")  
)
```

```
CHUNK_SIZE: int = int(os.getenv("ATTACHMENT_CHUNK_SIZE", "1000"))
```

```
EMBED_MODEL: str = os.getenv("BEDROCK_EMBED_MODEL")
```

```
# -----
```

```
# Lazy embedding client
```

```
# -----
```

```
_embeddings = None
```

```
def _get_embeddings():
```

```
    global _embeddings
```

```
    if _embeddings is None:
```

```
        from langchain_aws import BedrockEmbeddings
```

```

        region = os.getenv("AWS_DEFAULT_REGION")

        if not EMBED_MODEL:
            raise ValueError("BEDROCK_EMBED_MODEL environment variable not
set")

        _embeddings = BedrockEmbeddings(
            model_id=EMBED_MODEL,
            region_name=region,
        )
        return _embeddings

# -----
# Download
# -----

def download_attachment(url: str, email: str, token: str) -> bytes:
    response = requests.get(
        url,
        auth=HTTPBasicAuth(email, token),
        timeout=60,
    )
    response.raise_for_status()
    return response.content

# -----
# Hashing
# -----

def compute_file_hash(file_bytes: bytes) -> str:
    return hashlib.sha256(file_bytes).hexdigest()

def compute_combined_hash(file_hashes: list[str]) -> Optional[str]:
    if not file_hashes:
        return None
    combined = "|".join(sorted(file_hashes))
    return hashlib.sha256(combined.encode()).hexdigest()

# -----
# Text extraction
# -----

def extract_pdf_text(file_bytes: bytes) -> str:
    try:
        import pdfplumber

```

```

        full_text = ""
        with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
            for page in pdf.pages:
                full_text += (page.extract_text() or "") + "\n"
            return full_text.strip()
    except Exception as exc:
        logger.warning(f"[Attach] PDF extraction failed: {exc}")
        return ""

def extract_image_text(file_bytes: bytes) -> str:
    try:
        from PIL import Image
        import pytesseract

        image = Image.open(io.BytesIO(file_bytes))
        return pytesseract.image_to_string(image)
    except Exception as exc:
        logger.warning(f"[Attach] OCR failed: {exc}")
        return ""

def extract_text_file(file_bytes: bytes) -> str:
    return file_bytes.decode("utf-8", errors="ignore")

def extract_attachment_content(
    attachment: dict,
    email: str,
    token: str,
) -> tuple[bytes, str]:

    url = attachment.get("content", "")
    mime = attachment.get("mimeType", "").lower()
    filename = attachment.get("filename", "")

    try:
        file_bytes = download_attachment(url, email, token)
    except Exception as exc:
        logger.error(f"[Attach] Download failed for '{filename}': {exc}")
        return b"", ""

    if "pdf" in mime:
        text = extract_pdf_text(file_bytes)
    elif "image" in mime:
        text = extract_image_text(file_bytes)
    elif "text" in mime or filename.endswith((".txt", ".log", ".csv", ".md")):
        text = extract_text_file(file_bytes)

```

```

    else:
        text = file_bytes.decode("utf-8", errors="ignore")

    return file_bytes, text

# -----
# Sentence-aware semantic chunking
# -----

def chunk_text(text: str, chunk_size: int = CHUNK_SIZE) -> list[str]:
    """
    Sentence-aware chunking.
    Prevents cutting sentences mid-way.
    """
    sentences = re.split(r'(?<=[.!?]) +', text)
    chunks = []
    current_chunk = ""

    for sentence in sentences:
        if len(current_chunk) + len(sentence) <= chunk_size:
            current_chunk += sentence + " "
        else:
            if current_chunk:
                chunks.append(current_chunk.strip())
                current_chunk = sentence + " "

    if current_chunk:
        chunks.append(current_chunk.strip())

    return chunks

# -----
# Embeddings & similarity
# -----

def cosine_similarity(a: list[float], b: list[float]) -> float:
    va = np.array(a, dtype=np.float32)
    vb = np.array(b, dtype=np.float32)

    norm_a = np.linalg.norm(va)
    norm_b = np.linalg.norm(vb)

    if norm_a == 0 or norm_b == 0:
        return 0.0

    return float(np.dot(va, vb) / (norm_a * norm_b))

```

```

# -----
# Orchestrator: process all attachments for one ticket
# -----

def process_ticket_attachments(
    attachments: list[dict],
    email: str,
    token: str,
    ticket_context: str,
    tenant_id: str,
    ticket_key: str,
) -> dict:
    """
    Process all attachments for a ticket.

    Returns:
    {
        "total": int,
        "relevant": int,
        "irrelevant": int,
        "all_relevant": bool,
        "any_relevant": bool,
        "file_hashes": [str, ...],
        "combined_hash": str | None,
        "details": [...]
    }
    """

    from utils.db import get_attachment_hash, get_attachment_result,
    upsert_attachment

    results = []
    file_hashes = []

    for att in attachments:
        att_id = att.get("attachment_id", "")
        filename = att.get("filename", "unknown")

        # --- Extract ---
        file_bytes, extracted_text = extract_attachment_content(att, email,
token)

        if not file_bytes:
            results.append({
                "attachment_id": att_id,
                "filename": filename,
                "is_relevant": True, # fail-open
                "score": 0.0,

```

```

        "file_hash": "",
        "extracted_text": "",
    })
    continue

    current_file_hash = compute_file_hash(file_bytes)
    file_hashes.append(current_file_hash)

    # --- Cache check ---
    existing_hash = get_attachment_hash(tenant_id, att_id)

    if existing_hash and existing_hash == current_file_hash:
        cached = get_attachment_result(tenant_id, att_id)
        if cached:
            logger.info(f"[Attach] '{filename}' - cache hit.")
            results.append({
                "attachment_id": att_id,
                "filename": filename,
                "is_relevant": cached.get("is_relevant", True),
                "score": cached.get("relevance_score", 0.0),
                "file_hash": current_file_hash,
                "extracted_text": cached.get("extracted_text", ""),
            })
            continue

    # --- Relevance Check ---
    is_relevant, score = check_attachment_relevance(
        att,
        file_bytes,
        extracted_text,
        ticket_context,
    )

    # --- Persist ---
    try:
        upsert_attachment(
            tenant_id=tenant_id,
            ticket_key=ticket_key,
            attachment_id=att_id,
            filename=filename,
            mime_type=att.get("mimeType", ""),
            file_hash=current_file_hash,
            extracted_text=extracted_text,
            relevance_score=score,
            is_relevant=is_relevant,
        )
    except Exception as exc:
        logger.error(f"[Attach] DB upsert failed for '{filename}': {exc}")

```



```

        results.append({
            "attachment_id": att_id,
            "filename": filename,
            "is_relevant": is_relevant,
            "score": score,
            "file_hash": current_file_hash,
            "extracted_text": extracted_text,
        })

    combined_hash = compute_combined_hash(file_hashes)

    relevant_count = sum(1 for r in results if r["is_relevant"])
    irrelevant_count = len(results) - relevant_count

    return {
        "total": len(results),
        "relevant": relevant_count,
        "irrelevant": irrelevant_count,
        "all_relevant": irrelevant_count == 0,
        "any_relevant": relevant_count > 0,
        "file_hashes": file_hashes,
        "combined_hash": combined_hash,
        "details": results,
    }

# -----
# Relevance pipeline (UPDATED)
# -----

def check_attachment_relevance(
    attachment: dict,
    file_bytes: bytes,
    extracted_text: str,
    ticket_context: str,
) -> tuple[bool, float]:

    filename = attachment.get("filename", "?")

    if not extracted_text.strip():
        logger.info(f"[Attach] '{filename}' - empty extraction.")
        return False, 0.0

    if not ticket_context.strip():
        logger.warning(f"[Attach] '{filename}' - no ticket context.")
        return True, 1.0 # fail-open

    try:

```

```

embeddings = _get_embeddings()

# ◇ Embed ticket context as query
ticket_embedding = embeddings.embed_query(ticket_context)

# ◇ Chunk attachment semantically
chunks = chunk_text(extracted_text)

if not chunks:
    return False, 0.0

# ◇ Batch embed attachment chunks (more efficient)
chunk_embeddings = embeddings.embed_documents(chunks)

max_score = 0.0

for chunk_emb in chunk_embeddings:
    score = cosine_similarity(ticket_embedding, chunk_emb)
    if score > max_score:
        max_score = score

is_relevant = max_score >= RELEVANCE_THRESHOLD

logger.info(
    f"[Attach] '{filename}' - score={max_score:.4f}, "
    f"threshold={RELEVANCE_THRESHOLD}, relevant={is_relevant}"
)

return is_relevant, round(max_score, 4)

except Exception as exc:
    logger.error(f"[Attach] Relevance check failed for '{filename}': {exc}")
    return True, 0.0

```